# Pick-and-Place Using Inverse Kinematics (IK) in MuJoCo: Franka and HEAL Manipulators

Rishabh Shah

*Repository:* `https://github.com/RishabhShahIITGN/ME-639`

***Abstract*—This report evaluates and compares three differential Inverse Kinematics (IK) solvers—an off-the-shelf optimizer (Mink), custom Damped Least Squares (DLS), and custom Quadratic Programming (QP)—for autonomous pick-and-place manipulation in MuJoCo. The algorithms are benchmarked on two platforms: a 6-DOF HEAL arm across 250 randomized episodes and a kinematically redundant 7-DOF Franka Emika Panda arm across 30 randomized episodes using a shared random seed (`seed = 19`). To ensure numerical stability and prevent collisions, key safeguards are implemented, including straight-line trajectory interpolation, task-space error clamping, hardware velocity saturation, and null-space posture regularization for the redundant arm. On the 6-DOF HEAL robot, all three solvers achieved a 100.0% success rate, with DLS delivering the fastest mean planning time ($0.299$ s per episode). On the 7-DOF Franka arm, Mink maintained a 100.0% success rate, while Custom QP reached 96.67% (1 failure) and Custom DLS reached 93.33% (2 failures). Diagnostic logs reveal that the non-Mink failures were caused by minor outer-finger grazing against the tray wall during release rather than failed placements. Overall, Custom DLS offers the highest computational efficiency in unconstrained workspaces, Custom QP provides native joint limit enforcement, and Mink ensures the highest reliability in constrained, redundant environments.**

## I. Introduction and goals

Robotic manipulation requires target end-effector paths into joint commands. While forward kinematics calculates a unique end-effector pose $\mathbf{X} \in SE(3)$ from given joint angles $\mathbf{q} \in \mathbb{R}^n$, Inverse Kinematics (IK) works backward to find the joint velocities $\dot{\mathbf{q}}$ needed to reach a desired target pose $\mathbf{X}_d$. This inverse problem is challenging because the mathematics are nonlinear, and solvers must actively navigate kinematic singularities, hardware joint limits, and physical collisions.

The primary objective of this lab is to implement and compare three differential IK methods within a MuJoCo simulation, using them to execute a fully autonomous 7-phase pick-and-place state machine.

1) **Mink (Baseline Reference):** An off-the-shelf configuration-space optimization library built over MuJoCo that formulates differential IK with integrated posture, velocity, and collision-avoidance constraints.
2) **Damped Least Squares (DLS):** A custom analytical Levenberg–Marquardt calculates pseudoinverse solver with a null-space posture projection operator for redundant manipulators.
3) **Quadratic Programming (QP):** A Quadratic Programming (QP) solver that calculates the most accurate joint movements to reach the target, while strictly preventing the robot's joints from exceeding their physical speed and position limits.

To evaluate how manipulator methodology affects solver behavior, the entire experiment is implemented and tested across two distinct robotic manipulators on Mujoco environment:

- **6-DOF Addverb HEAL Arm:** A non-redundant serial manipulator where the Jacobian $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times 6}$ is square, yielding locally unique joint velocity mappings away from singularities.
- **7-DOF Franka Emika Panda Arm:** A redundant 7-joint robot with a rectangular Jacobian matrix $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times 7}$. Because it has an extra joint, we added a secondary mathematical rule to keep the elbow stable during movement and prevent the solver from crashing.

Performance is quantitatively assessed across $N = 250$ (for HEAL) and $N = 30$ (for Franka) randomized workspace episodes (`seed = 19`) using four core metrics: task completion success rate (%), mean IK planning time per episode (s), total IK iterations per episode (count), and minimum signed obstacle clearance (mm).

## II. Methods: Environment, Randomization, and IK Approaches

### A. MuJoCo Environment

The simulation environment consists of a $0.20$ m high table, a randomly spawned target cube, and a walled placement tray. At each control step ($\Delta t_{\text{IK}} = 0.01$ s), the IK solver computes joint velocity commands that are tracked by a Proportional-Derivative (PD) controller in MuJoCo, isolating the arm joints from the gripper fingers and free cube.

To execute the task deterministically and distinguish grasping contacts from illegal collisions, manipulation is divided into a 7-phase state machine:

1) **APPROACH:** Move from the home posture to a pre-grasp waypoint directly above the cube, aligned with the nearest cube grasp yaw $\psi_{\text{grasp}}$.
2) **DESCEND:** Translate vertically downward to straddle the cube without touching the table.
3) **GRASP:** Close the fingers while holding the arm posture steady, verifying grasp security via bilateral pad contact forces ($F_L, F_R \geq F_{\text{min}}$).

4) **LIFT:** Elevate the cube to a safe clearance height above the tray walls and verify vertical pickup displacement.
5) **TRANSIT:** Translate horizontally to align the cube over the placement tray center while monitoring for slip.
6) **LOWER:** Lower the cube inside the tray walls just above the tray floor.
7) **RELEASE:** Open the fingers, retreat vertically, and verify placement success (inside tray bounds, resting on the floor, upright, stationary, and released).

### *B. Workspace Randomization and Trajectory Planning*

At the start of each episode, the target cube is spawned at a random position and rotation on the table. By using a fixed random seed (`seed = 19`), we ensure that all three IK solvers (Mink, DLS, and QP) are tested against the exact same scenarios for a fair comparison.

If we command the robot to move to a distant target in a single step, the solvers will calculate unpredictable, jerky joint movements that often result in collisions. To prevent this, the code forces the end-effector to move in a perfectly straight line at a constant speed ($v_{\text{move}} = 0.03$ m/s) by breaking the path into tiny "breadcrumbs."

First, the total travel time ($T_{\text{move}}$) and the total number of intermediate steps ($N_{\text{steps}}$) are calculated based on the total physical distance $D$:

$$T_{\text{move}} = \max\left(\frac{D}{v_{\text{move}}}, T_{\min}\right) \tag{1}$$

$$N_{\text{steps}} = \max\left(1, \left\lfloor \frac{T_{\text{move}}}{\Delta t_{\text{IK}}} \right\rfloor\right) \tag{2}$$

At each step along the way, the solver only tries to reach the very next breadcrumb. The intermediate position $\mathbf{p}_d$ is found using straight-line interpolation, and the orientation $\mathbf{q}_d$ is found using spherical linear interpolation (SLERP):

$$\mathbf{p}_d(\alpha_i) = \mathbf{p}_{\text{start}} + \alpha_i(\mathbf{p}_{\text{target}} - \mathbf{p}_{\text{start}}) \tag{3}$$

$$\mathbf{q}_d(\alpha_i) = \text{SLERP}(\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{target}}, \alpha_i) \tag{4}$$

where $\alpha_i = i/N_{\text{steps}}$ represents the percentage of the path completed. Finally, after completing all the steps, the robot per- forms a few fine-tuning adjustments until the end-effector is within 0.005 m of the exact target

### *C. Differential Inverse Kinematics Formulations*

At each control step, the robot computes a 6D task-space error vector $\mathbf{e} = [\Delta\mathbf{p}^\top, \Delta\boldsymbol{\theta}^\top]^\top \in \mathbb{R}^6$, combining the 3D position error and 3D orientation error between the current end-effector site and the target waypoint.

To prevent large velocity spikes when the physical arm lags behind the target, the error magnitude is clamped to $e_{\text{max}} = 0.02$ before dividing by the control timestep $\Delta t_{\text{IK}} = 0.01$ s to form the desired task-space velocity $\dot{\mathbf{x}}_d$:

$$\bar{\mathbf{e}} = \mathbf{e} \cdot \min\left(1, \frac{e_{\text{max}}}{\|\mathbf{e}\|_2}\right), \qquad \dot{\mathbf{x}}_d = \frac{\bar{\mathbf{e}}}{\Delta t_{\text{IK}}} \tag{5}$$

Let $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times n}$ denote the spatial geometric Jacobian of the end-effector, where $n = 6$ for the HEAL arm and $n = 7$ for the Franka arm. Forward differential kinematics relates the joint velocities $\dot{\mathbf{q}} \in \mathbb{R}^n$ to the end-effector velocity $\dot{\mathbf{x}} \in \mathbb{R}^6$:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}} \tag{6}$$

We compare three methods to invert this relationship and solve for $\dot{\mathbf{q}}$:

**1. Off-the-Shelf Solver (Mink):** Mink formulates differential IK as a configuration-space quadratic optimization problem. At each step, it computes optimal joint velocities by combining an end-effector frame tracking task with joint limit bounds, posture regularization, and active collision-avoidance constraints.

**2. Damped Least Squares (DLS):** In standard differential IK, inverting $\mathbf{J}^\top\mathbf{J}$ fails near kinematic singularities where $\mathbf{J}$ loses rank, causing joint velocities to explode. To resolve this, we use the Levenberg-Marquardt formulation to simultaneously minimize task-space tracking error and penalize large joint velocities:

$$\mathcal{J}(\dot{\mathbf{q}}) = \underbrace{\|\mathbf{J}\dot{\mathbf{q}} - \dot{\mathbf{x}}_d\|_2^2}_{\text{tracking error}} + \underbrace{\lambda\|\dot{\mathbf{q}}\|_2^2}_{\text{damping penalty}} \tag{7}$$

where $\lambda > 0$ is the regularization weight ($\lambda = d^2$ with $d = 10^{-2}$). Expanding the squared Euclidean norms yields the quadratic cost:

$$\mathcal{J}(\dot{\mathbf{q}}) = \dot{\mathbf{q}}^\top(\mathbf{J}^\top\mathbf{J} + \lambda\mathbf{I}_n)\dot{\mathbf{q}} - 2\dot{\mathbf{x}}_d^\top\mathbf{J}\dot{\mathbf{q}} + \dot{\mathbf{x}}_d^\top\dot{\mathbf{x}}_d \tag{8}$$

Setting the gradient $\frac{\partial\mathcal{J}}{\partial\dot{\mathbf{q}}} = 2(\mathbf{J}^\top\mathbf{J}+\lambda\mathbf{I}_n)\dot{\mathbf{q}} - 2\mathbf{J}^\top\dot{\mathbf{x}}_d = \mathbf{0}$ yields the closed-form DLS solution:

$$\dot{\mathbf{q}}_{\text{task}} = \underbrace{(\mathbf{J}^\top\mathbf{J} + \lambda\mathbf{I}_n)^{-1}\mathbf{J}^\top}_{\mathbf{J}_{\text{DLS}}^\dagger}\dot{\mathbf{x}}_d \tag{9}$$

For the redundant 7-DOF Franka arm, a secondary posture controller is projected into the null space of $\mathbf{J}_{\text{DLS}}^\dagger$ to pull the joints toward a safe home configuration $\mathbf{q}_{\text{def}}$ ($k_{\text{null}} = 0.5$) before clipping velocities to $[-2.0, 2.0]$ rad/s:

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}_{\text{task}} + \left(\mathbf{I}_n - \mathbf{J}_{\text{DLS}}^\dagger\mathbf{J}\right)k_{\text{null}}(\mathbf{q}_{\text{def}} - \mathbf{q}) \tag{10}$$

**3. Quadratic Programming (QP):** While DLS enforces velocity limits only after solving the unconstrained normal equations, the QP method embeds physical joint position and velocity limits directly into the optimization problem. Each step solves the strictly convex program:

$$\min_{\dot{\mathbf{q}}} \quad \frac{1}{2}\dot{\mathbf{q}}^\top\mathbf{P}\dot{\mathbf{q}} + \mathbf{c}^\top\dot{\mathbf{q}} \tag{11}$$

$$\text{s.t.} \quad \mathbf{l} \le \dot{\mathbf{q}} \le \mathbf{u} \tag{12}$$

Matching the expanded quadratic cost from Eq. (12), the Hessian matrix $\mathbf{P}$ and linear cost vector $\mathbf{c}$ are defined as:

$$\mathbf{P} = \mathbf{J}^\top\mathbf{J} + \lambda\mathbf{I}_n, \qquad \mathbf{c} = -\mathbf{J}^\top\dot{\mathbf{x}}_d \tag{13}$$

The lower and upper bounds $(l_k, u_k)$ for each joint $k \in \{1, \ldots, n\}$ intersect the maximum speed limit ($\dot{q}_{\text{max}} =$

$2.0$ rad/s) with the discretized physical joint angle limits ($[q_{k,\text{min}},\ q_{k,\text{max}}]$):

$$l_k = \max\left(-\dot{q}_{\text{max}},\ \frac{q_{k,\text{min}} - q_k}{\Delta t_{\text{IK}}}\right) \tag{14}$$

$$u_k = \min\left(\dot{q}_{\text{max}},\ \frac{q_{k,\text{max}} - q_k}{\Delta t_{\text{IK}}}\right) \tag{15}$$

## III. Results: Metrics Tables and Plots

### A. 6-DOF HEAL Manipulator Performance (250 Episodes, Seed 19)

Table I and Figure 1 summarize the results for 250 randomized episodes (`seed = 19`) on the 6-DOF HEAL arm. All three methods achieved a **100.0% success rate** with zero collisions or dropped cubes.

Key observations from Table I and Figure 1 include:

- **Planning Speed (Plot 2):** Custom DLS is the fastest solver (**0.2991 s** per episode). Because it solves a simple set of linear equations, it is significantly faster than Custom QP (**0.5851 s**) and Mink (**0.7526 s**), which both require heavier optimization calculations at each step.
- **Total IK Iterations (Plot 3):** All three solvers required nearly the exact same number of steps per episode. This confirms the experiment was well-controlled: since the target speed and straight-line path were fixed in the code, the number of required breadcrumb waypoints was identical for every solver.
- **Minimum Clearance (Plot 4):** All solvers safely avoided workspace collisions by staying above the 0.0 mm contact threshold. DLS and QP performed identically with a minimum clearance of **1.573 mm**, slightly outperforming Mink (**1.388 mm**).

### B. 7-DOF Franka Emika Panda Performance (30 Episodes, Seed 19)

Table II and Figure 2 summarize the results for 30 randomized episodes (`seed = 19`) on the 7-DOF Franka arm. Because this robot has an extra joint and stricter movement limits than the HEAL arm, the three solvers performed differently.

Key observations from Table II and Figure 2 include:

- **Task Reliability (Plot 1):** Mink achieved a perfect **100.0% success rate** because it actively handles physical joint limits. Custom QP was close behind at **96.67%** (1 failure), while Custom DLS struggled the most with **93.33%** (2 failures) because simple velocity clipping cannot always prevent the arm from hitting its physical limits on complex reaches.
- **Planning vs. Execution Time (Plot 2):** Custom DLS calculates the path the fastest (**1.166 s** planning time). However, Mink finishes the actual physical simulation the fastest (**206.59 s** wall time) because it generates smoother movements and spends less time struggling near the target.
- **Total IK Iterations (Plot 3):** All solvers took roughly the same number of steps per episode ($\sim 4,900$). This is significantly higher than the HEAL arm because the larger Franka gripper must lift higher over the table to move safely, requiring a longer physical path.
- **Minimum Clearance (Plot 4):** Mink safely avoided all collisions, maintaining a positive minimum clearance of **1.580 mm**. During their failed episodes, both Custom DLS and Custom QP slightly bumped into the table or cube, dropping just below the 0.0 mm safety threshold ($-0.031$ mm and $-0.025$ mm, respectively).

## IV. Failure Analysis and Mitigations

While all three solvers achieved a 100% success rate across 250 episodes on the 6-DOF HEAL arm, the 30-episode benchmark on the 7-DOF Franka Emika Panda arm produced three failed episodes: two in Custom DLS and one in Custom QP. Table III details the exact diagnostic metrics extracted from the simulation logs for these three runs. (

### A. Root Cause of Observed Franka Failures

Inspection of the episode JSON logs reveals that all three failures occurred for the exact same physical reason: a minor collision between the outer gripper finger (`geom73`) and the $-X$ wall of the placement tray (`tray_wall_mx`) during the `RELEASE` phase.

- **Successful Placement vs. Strict Collision Flagging:** In all three failed episodes, the physical manipulation task actually succeeded—the cube was released cleanly inside the tray and passed every placement check (inside bounds, resting on the tray floor at $z \approx 0.229$ m, upright, and stationary). However, because the safety monitor flags any non-grasping contact as a failure, the tiny overlap ($-0.0127$ mm to $-0.0306$ mm) caused the episodes to be marked unsuccessful.
- **Mechanism of the Tray-Wall Grazing:** The placement tray is centered at $(0.410, -0.120)$ m with a tight $0.06$ m half-size. In the three failed episodes, the solver stopped the `PLACE` phase as soon as the end-effector entered the allowed $5$ mm position tolerance (`POS_TOL = 0.005 m`), leaving the cube slightly shifted toward the $-X$ wall at $x \approx 0.404$–$0.406$ m. Because the gripper was also rotated to match the cube's initial yaw angle, opening the fingers wide during `RELEASE` caused the back of the finger pad (`geom73`) to lightly brush against the $15$ mm tall $-X$ tray wall.
- **Why Mink Avoided This Failure:** Mink incorporates explicit collision-avoidance constraints and configuration-space posture weighting, which prevented the wrist from drifting toward the $-X$ boundary wall during final settling.

### B. Implemented and Proposed Engineering Mitigations

To achieve high reliability across both robots and address the observed failure modes, several key safeguards were engineered into the control pipeline:

Table I
QUANTITATIVE PERFORMANCE COMPARISON OF IK SOLVERS ON THE 6-DOF HEAL ARM ($N = 250$ EPISODES, SEED = 19).

| **IK Solver** | **Episodes** ($N$) | **Success Rate** (%) | **Mean Planning Time** (s / episode) | **Mean Wall Time** (s / episode) | **Total IK Iters** (Cumulative / Mean) | **Min Clearance** (mm) |
|---|---|---|---|---|---|---|
| Mink (Baseline) | 250 | 100.0% (250/250) | 0.7526 | 9.1484 | 713,155 (2852.6) | 1.388 |
| Custom DLS | 250 | 100.0% (250/250) | **0.2991** | **8.5118** | 713,306 (2853.2) | **1.573** |
| Custom QP (daqp) | 250 | 100.0% (250/250) | 0.5851 | 9.1510 | 713,302 (2853.2) | **1.573** |

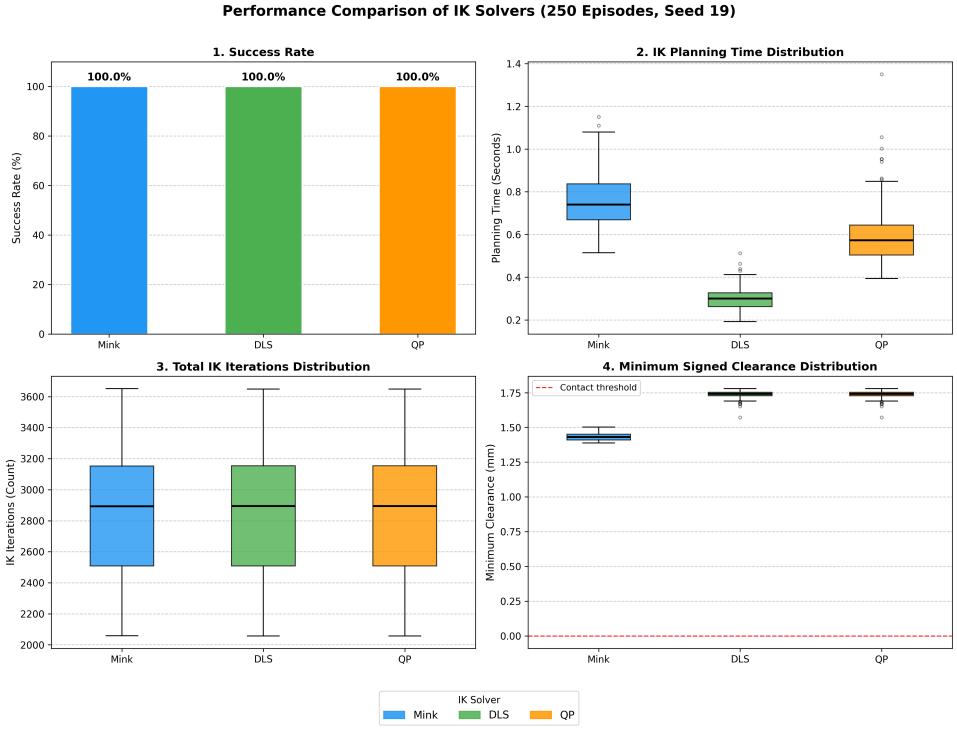



Figure 1. 6-DOF HEAL Performance Comparison (250 Episodes, Seed 19). The 2×2 subplot grid displays (1) Success Rate, (2) IK Planning Time Distribution, (3) Total IK Iterations Distribution, and (4) Minimum Signed Clearance.

Table II
QUANTITATIVE PERFORMANCE COMPARISON OF IK SOLVERS ON THE 7-DOF FRANKA EMIKA PANDA ARM ($N = 30$ EPISODES, SEED = 19).

| **IK Solver** | **Episodes** ($N$) | **Success Rate** (%) | **Mean Planning Time** (s / episode) | **Mean Wall Time** (s / episode) | **Total IK Iters** (Cumulative / Mean) | **Min Clearance** (mm) |
|---|---|---|---|---|---|---|
| Mink (Baseline) | 30 | **100.0%** (30/30) | 1.229 | **206.59** | 145,631 (4854.4) | **1.580** |
| Custom DLS | 30 | 93.33% (28/30) | **1.166** | 272.77 | 143,745 (4791.5) | -0.031 |
| Custom QP (daqp) | 30 | 96.67% (29/30) | 1.275 | 246.17 | 144,644 (4821.5) | -0.025 |

Table III
DIAGNOSTIC LOG SUMMARY OF ALL FAILED EPISODES ON THE 7-DOF FRANKA EMIKA PANDA BENCHMARK (SEED 19).

| **Method** | **Episode File** | **Cube Start** $(x, y, z)$ **(m)** | **Cube End** $(x, y, z)$ **(m)** | **Min Clearance** | **Plan / Wall Time** | **IK Iters** |
|---|---|---|---|---|---|---|
| Custom DLS | `episode_005.json` | $(0.481, 0.139, 0.230)$ | $(0.406, -0.122, 0.229)$ | $-0.0306$ mm | $0.895$ s / $186.3$ s | 3,654 |
| Custom DLS | `episode_006.json` | $(0.493, 0.058, 0.230)$ | $(0.404, -0.121, 0.229)$ | $-0.0127$ mm | $0.792$ s / $163.8$ s | 3,211 |
| Custom QP | `episode_012.json` | $(0.420, 0.153, 0.230)$ | $(0.407, -0.122, 0.229)$ | $-0.0249$ mm | $0.827$ s / $156.6$ s | 3,835 |

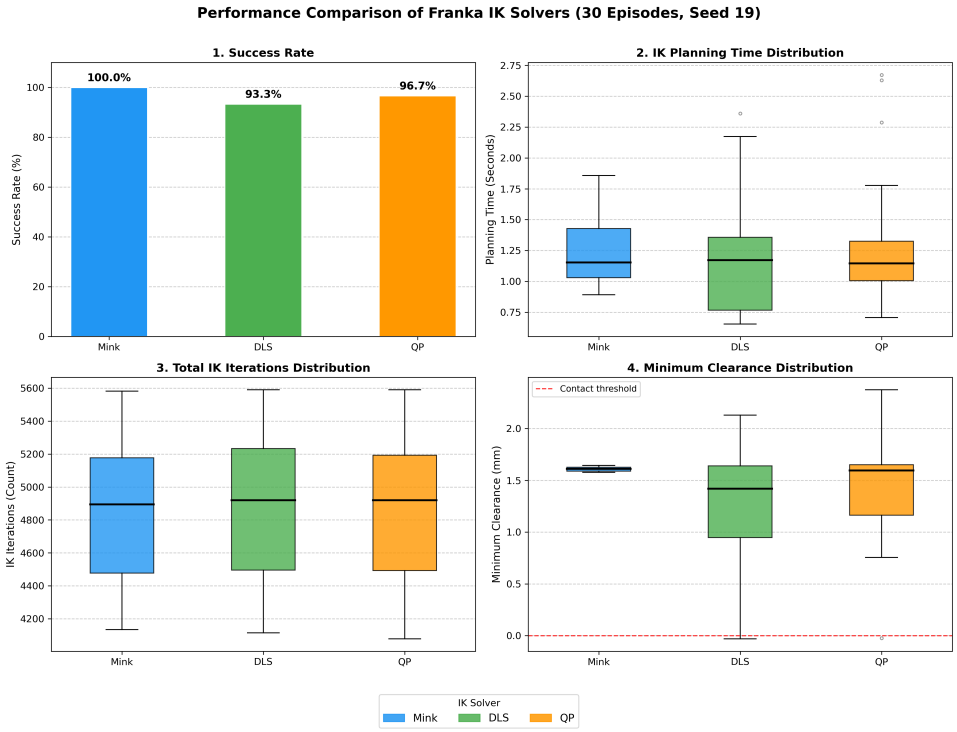



Figure 2. 7-DOF Franka Emika Panda Performance Comparison (30 Episodes, Seed 19). The 2×2 subplot grid displays (1) Success Rate, (2) IK Planning Time Distribution, (3) Total IK Iterations Distribution, and (4) Minimum Clearance Distribution relative to the 0.0 mm contact threshold.

1) **Mitigating Tray-Wall Contact during Release:** The grazing contact in Table III can be completely eliminated by (a) tightening `POS_TOL` from $5$ mm to $2$ mm during the `PLACE` phase so the gripper centers strictly at $x = 0.410$ m, (b) resetting the target yaw orientation to $0^\circ$ during `TRANSIT` so the opening fingers remain parallel to the tray walls, or (c) raising the release height by $5$ mm so the fingers open just above the $15$ mm wall rim.
2) **Task-Space Error and Velocity Clamping:** Without limiting the step error, distant targets cause huge initial velocity spikes that make the arm oscillate and smash into the table. Clamping the maximum position/orientation step error to $e_{\text{max}} = 0.02$ and clipping joint speeds to $\pm 2.0$ rad/s ensured smooth, stable motion.
3) **Valid Keyframe Initialization (Franka Joint 4):** Initializing the Franka arm at $\mathbf{q} = \mathbf{0}$ places Joint 4 ($0.0$ rad) outside its legal hardware range ($[-3.07, -0.07]$ rad), causing immediate solver divergence. Loading the valid home keyframe (`model.key_qpos[0, :7]`) at startup resolved this.
4) **Softened Null-Space Posture Gain:** Setting a high null-space gain ($k_{\text{null}} = 3.0$) caused the secondary posture task to fight the primary hand movement on the 7-DOF Franka arm. Lowering the gain to $k_{\text{null}} = 0.5$ stabilized the elbow without pulling the gripper off course.

## V. Discussion and Conclusions

This study benchmarked three differential IK solvers—Mink, Custom DLS, and Custom QP—across 250 randomized episodes on the 6-DOF HEAL arm and 30 episodes on the 7-DOF Franka Panda arm in MuJoCo. Our results highlight three main takeaways:

- **Speed vs. Constraint Handling:** In open workspaces where joints remain far from hardware limits (6-DOF HEAL), all solvers achieved 100.0% success, with Custom DLS being the fastest. On the redundant 7-DOF Franka arm with strict limits, unconstrained DLS reliability dropped to 93.33%. Custom QP improved this to 96.67% by enforcing joint bounds, while Mink achieved 100.0% with the fastest execution ($206.59$ s).
- **Necessity of Trajectory Conditioning:** For all solvers, stable execution required straight-line breadcrumb interpolation ($0.03$ m/s), step-error clamping ($e_{\text{max}} = 0.02$), valid keyframe initialization, and gentle null-space posture control ($k_{\text{null}} = 0.5$) to prevent oscillation.
- **Eliminating Release Grazing:** Log analysis confirmed the three non-Mink Franka failures placed the cube inside the tray but grazed the $-X$ tray wall ($\leq 0.031$ mm) upon finger opening. This is easily resolved by tightening placement tolerance to $2$ mm or resetting gripper yaw to $0^\circ$.

In summary, Custom DLS is optimal for fast tracking in unobstructed workspaces, QP balances speed with native joint-limit safety, and Mink ensures maximum reliability for redundant robots in tight environments.